

Python y venv

- 1. Necesidad de entornos virtuales
- 2. Paso 1: crear el entorno virtual
- 3. Paso 2: activar el entorno virtual
- 4. Paso 3: Instalar un módulo
- 5. Paso 4: Salir del entorno
- 6. Uso del fichero requirements
- 7. Borrar el entorno virtual
- 8. Ejemplos de uso de entornos virtuales
 - 8.1. FastAPI
 - 8.2. El módulo requests
 - 8.3. Jupyter Notebook
- 9. Enlaces

1. Necesidad de entornos virtuales

Puede consultar detalles sobre entornos virtuales en este enlace: [A Guide to Python's Virtual Environments](#)

Pero básicamente se pretende:

- Poder trabajar con proyectos con distintas versiones de lenguaje de programación.
- Poder tener en cada proyecto distintas versiones de los módulos que se importen.
- Poder reproducir el entorno de trabajo en otro equipo a partir de un fichero de texto con los requisitos.
- No añadir a la configuración global del equipo módulos que son específicos de cada proyecto.

2. Paso 1: crear el entorno virtual

Cree una carpeta para el proyecto y sitúese en ella:

```
clases@d12:~$ mkdir proyecto
clases@d12:~$ cd proyecto/
clases@d12:~/proyecto$
```

El comando para crear un entorno virtual en Python es `python3 -m venv <nombre_directorio_virtual>`.

Es posible que al intentar crear el entorno virtual le solicite instalar el módulo venv si no estuviese instalado.

```
On Debian/Ubuntu systems, you need to install the python3-venv
package using the following command.
```

```
sudo apt install python3.11-venv
```

```
Para ello: `` clases@d12:~/proyecto$ sudo apt install python3.11-venv [sudo] contraseña para clases:
Leyendo lista de paquetes... Hecho Creando árbol de dependencias... Hecho
```

```
....

Configurando python3.11-venv (3.11.2-6) ...
clases@d12:~/proyecto$
``
```

Es frecuente darle al directorio donde se almacena toda la información del entorno virtual siempre el mismo nombre, por ejemplo en este caso **venv** (o **.venv**).

```
clases@d12:~/proyecto$ python3 -m venv venv
clases@d12:~/proyecto$ ls -l
total 4
drwxr-xr-x 5 clases clases 4096 dic  1 20:12 venv
clases@d12:~/proyecto$
```

3. Paso 2: activar el entorno virtual

Crear el entorno no implica que se esté usando. Para usarlo debe:

```
clases@d12:~/proyecto$ source venv/bin/activate
(venv) clases@d12:~/proyecto$
```

Tenga en cuenta que la activación desde un entorno Windows es diferente:

```
c:\proyecto\> .venv\Scripts\activate
```

Fuente: https://dev.to/e_farach/setting-up-a-python-virtual-environment-on-windows-for-absolute-beginners-il6

4. Paso 3: Instalar un módulo

En este ejemplo que resuelve un sistema de ecuaciones se usa el módulo **numpy** que no está instalado por defecto.

Este sería el contenido del fichero **main.py**:

```
import numpy as np

# solve(a, b) : Devuelve la solución del sistema de ecuaciones lineales
# con los coeficientes de la matriz a y los términos independientes
# de la matriz b.
# Sistema de dos ecuaciones y dos incógnitas
#   x  + 2y = 1
#   3x + 5y = 2
a = np.array([[1, 2], [3, 5]])
b = np.array([1, 2])
print(np.linalg.solve(a, b))
# Resultado [-1.  1.]
```

Recien creado el entorno no debe haber ningún módulo instalado por el usuario. El comando `pip freeze` no muestra nada a la salida:

```
(venv) clases@d12:~/proyecto$ pip freeze
(venv) clases@d12:~/proyecto$
```

Si ejecuta el ejemcódigo no funciona:

```
(venv) clases@d12:~/proyecto$ python3 main.py
Traceback (most recent call last):
  File "/home/usuario/CIBER-PPS/bloque_03_programacion_segura/docs/tema_01_venv/src/numpy/main.py", line
14, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
(venv) clases@d12:~/proyecto$
```

Instale el modulo necesario `numpy` en el entorno virtual con `pip`:

```
(venv) clases@d12:~/proyecto$ pip install numpy
Collecting numpy
...
Successfully installed numpy-2.2.0
(venv) clases@d12:~/proyecto$ pip freeze
numpy==2.2.0
(venv) clases@d12:~/proyecto$
```

Ahora puede probar de nuevo el ejemplo. El resultado será similar a

```
(venv) clases@d12:~/proyecto$ python3 main.py
[-1.  1.]
(venv) clases@d12:~/proyecto$
```

Nota: comprueba ahora la salida del comando `pip freeze`

5. Paso 4: Salir del entorno

Para salir del entorno se usa el comando `deactivate`:

```
(venv) clases@d12:~/proyecto$ deactivate
clases@d12:~/proyecto$
```

Puede intentar ejecutar el script anterior y observar que no funciona:

```
clases@d12:~/proyecto$ python3 main.py
Traceback (most recent call last):
  File "/home/usuario/CIBER-PPS/bloque_03_programacion_segura/docs/tema_01_venv/src/numpy/main.py", line 1, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
clases@d12:~/proyecto$
```

Si activa de nuevo el entorno se mantienen los módulos instalados previamente y puede ejecutar de nuevo el script, etc.

```
clases@d12:~/proyecto$ source venv/bin/activate
(venv) clases@d12:~/proyecto$ python3 main.py
[-1.  1.]
(venv) clases@d12:~/proyecto$
```

6. Uso del fichero requirements

Si quiere almacenar los requisitos del proyecto puede generar un fichero con el listado de los módulos (y sus versiones) usando `pip freeze` y redirigir la salida a fichero:

```
(venv) clases@d12:~/proyecto$ pip freeze > requirements.txt
(venv) clases@d12:~/proyecto$
```

Si quiere trabajar en otro equipo y recrear los mismos módulos basta, una vez creado y activado el nuevo entorno, ejecutar la instalación con **pip** de esos módulos a partir del fichero **requirements.txt** con el comando **pip3 install -r requirements.txt**

```
clases@d12:~/proyecto-sedeA$ ls -a
.  ..  requirements.txt
clases@d12:~/proyecto-sedeA$
clases@d12:~/proyecto-sedeA$ python3 -m venv venv
clases@d12:~/proyecto-sedeA$ source venv/bin/activate
(venv) clases@d12:~/proyecto-sedeA$ pip freeze
(venv) clases@d12:~/proyecto-sedeA$
(venv) clases@d12:~/proyecto-sedeA$ pip install -r requirements.txt
Collecting numpy==2.2.0
  Using cached numpy-2.2.0-cp311-cp311-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.4 MB)
Installing collected packages: numpy
Successfully installed numpy-2.2.0
(venv) clases@d12:~/proyecto-sedeA$
```

7. Borrar el entorno virtual

Si desea borrar el entorno virtual basta con borrar el directorio **venv** creado dentro del proyecto

```
(venv) clases@d12:~/proyecto$ deactivate
clases@d12:~/proyecto$ rm -rf venv/
clases@d12:~/proyecto$
```

8. Ejemplos de uso de entornos virtuales

8.1. FastAPI

En temas posteriores trabajará con el framework de Python FastAPI. En ese caso necesita el módulo **fastapi** y adicionalmente un servidor web como **uvicorn**. En este ejemplo se usa el fichero **main.py**

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

El proceso es similar al presentado previamente: crear un entorno virtual, iniciarlo e instalar los módulos:

```
clases@d12:~$ mkdir app
clases@d12:~$ cd app
clases@d12:~/app$ python3 -m venv venv
clases@d12:~/app$ source venv/bin/activate
```

```
(venv) clases@d12:~/app$ nano main.py
(venv) clases@d12:~/app$
```

Los dos módulos necesarios son **fastapi** y **uvicorn**

```
(venv) clases@d12:~/app$ pip freeze
(venv) clases@d12:~/app$ pip install fastapi
Collecting fastapi
...
Successfully installed annotated-types-0.6.0 anyio-3.7.1 fastapi-0.104.1
idna-3.6 pydantic-2.5.2 pydantic-core-2.14.5 sniffio-1.3.0 starlette-0.27.0
typing-extensions-4.8.0
(venv) clases@d12:~/app$ pip install uvicorn
Collecting uvicorn
...
Installing collected packages: h11, click, uvicorn
Successfully installed click-8.1.7 h11-0.14.0 uvicorn-0.24.0.post1
(venv) clases@d12:~/app$
```

Lance el servidor **uvicorn** con la aplicación. Observe la forma de invocarla:

```
(venv) clases@d12:~/app$ uvicorn main:app --reload
INFO:      Will watch for changes in these directories: ['/home/clases/app']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [3380] using StatReload
INFO:      Started server process [3382]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Y ya puede acceder a la app en la URL <http://127.0.0.1:8000> (o a <http://127.0.0.1:8000/docs> para ver la documentación de la API creada.)

```
clases@d12:~/app$ curl localhost:8000
{"message":"Hello World"}
clases@d12:~/app$ firefox localhost:8000
```

8.2. El módulo requests

Este modulo es posible que ya esté instalado, En ese caso solo es necesario importarlo y usarlo. Veamos un ejemplo sencillo de su uso:

```
import requests

URL = "https://www.uca.es" # "https://www.uca.es/noexiste"
res = requests.get(URL)

print(res)
print(res.headers)
# print(res.text)
```

El resultado será similar a

```
(venv) clases@d12:~/proyecto$ python prueba_request.py
<Response [200]>
{'Server': 'nginx', 'Date': 'Fri, 01 Dec 2023 19:47:02 GMT', 'Content-Type':
'text/html; charset=UTF-8', 'Content-Length': '20584', 'Connection': 'keep-
alive', 'X-Frame-Options': 'SAMEORIGIN', 'Link': '<https://www.uca.es/wp-
json/>; rel="https://api.w.org/"', 'cache-control': 'max-age=0, must-
revalidate', 'Content-Encoding': 'gzip', 'Vary': 'Accept-Encoding', 'X-
Forwarded-Proto': 'https', 'age': '0', 'Accept-Ranges': 'bytes'}
(venv) clases@d12:~/proyecto$
```

8.3. Jupyter Notebook

Proceso similar: crear un nuevo directorio, un entorno virtual, se activa y se instala el módulo `notebook`. Tiene más información sobre esta herramienta en <https://jupyter.org/>

En este ejemplo se mostrará Jupyter Notebook. También puede probar Jupyter Lab

Este ejemplo muestra que un módulo puede tener dependencias de otros y en este caso instala todos los necesarios (en este caso alrededor de 90):

```
clases@d12:~$ mkdir notebook && cd notebook
clases@d12:~/notebook$ python3 -m venv venv
clases@d12:~/notebook$ source venv/bin/activate
(venv) clases@d12:~/notebook$ pip install notebook
Collecting notebook
....
clases@d12:~/notebook$ pip freeze | wc -l
90
(venv) clases@d12:~/notebook$
```

Una vez instalado, lanzar la aplicación con el comando `jupyter notebook`

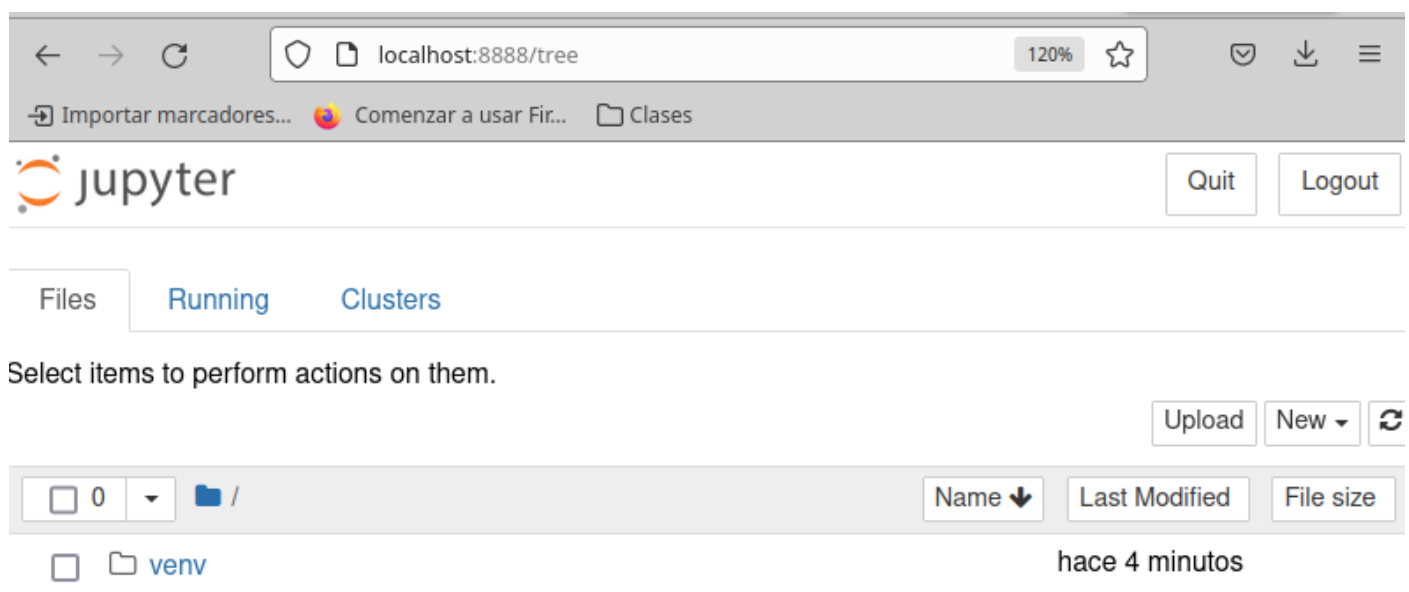
```
(venv) clases@d12:~/notebook$ jupyter notebook
[I 2023-12-01 22:00:04.835 ServerApp] Package notebook took 0.0000s to import
[I 2023-12-01 22:00:04.842 ServerApp] Package jupyter_lsp took 0.0066s to
import

....

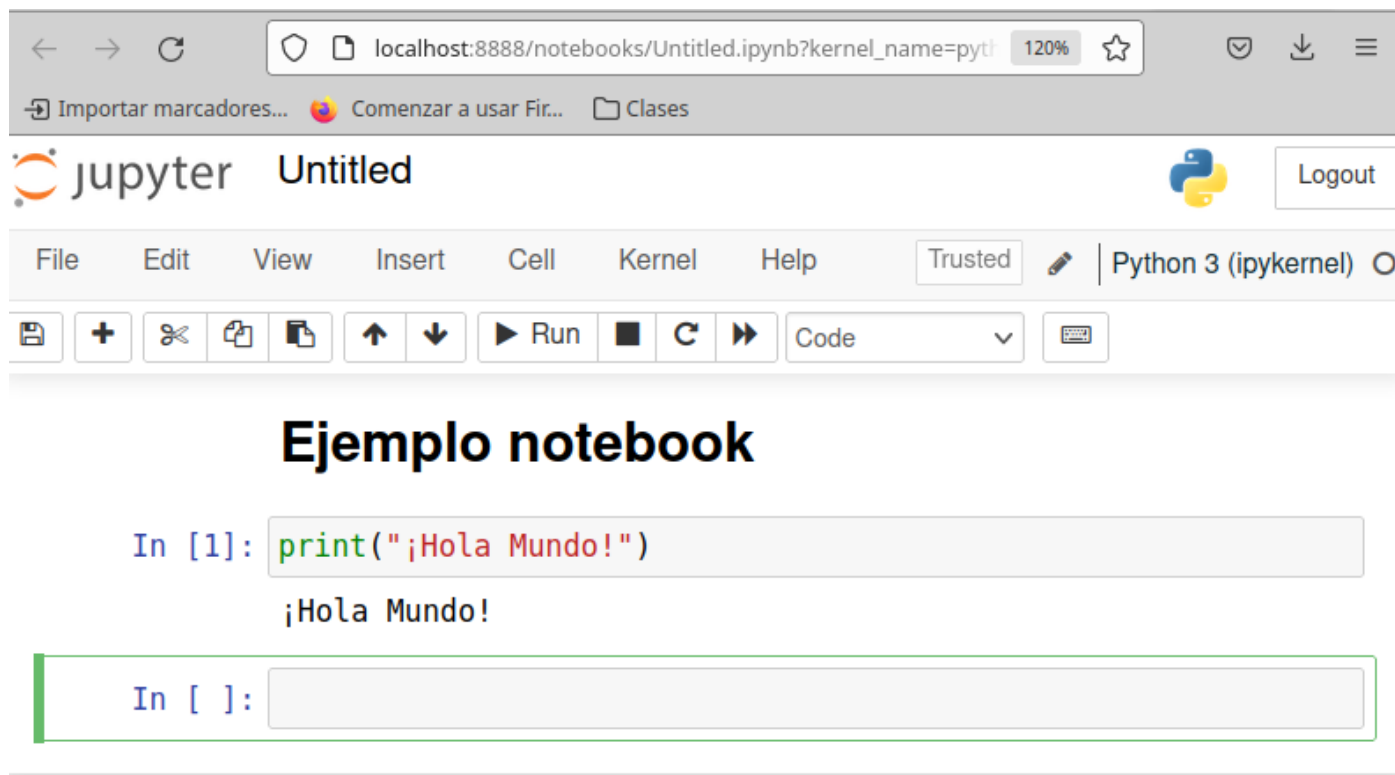
[I 2023-12-01 22:00:05.071 ServerApp] Serving notebooks from local directory:
/home/clases/notebook
[I 2023-12-01 22:00:05.071 ServerApp] Jupyter Server 2.11.1 is running at:
[I 2023-12-01 22:00:05.071 ServerApp] http://localhost:8888/tree?
token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
[I 2023-12-01 22:00:05.071 ServerApp] http://127.0.0.1:8888/tree?
token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
[I 2023-12-01 22:00:05.071 ServerApp] Use Control-C to stop this server and
shut down all kernels (twice to skip confirmation).
[C 2023-12-01 22:00:05.084 ServerApp]

To access the server, open this file in a browser:
file:///home/clases/.local/share/jupyter/runtime/jpserver-2652-
open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?
token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
http://127.0.0.1:8888/tree?
token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
```

Nos muestra los mensajes para poder acceder a Jupyter Notebook, aunque lo normal es que nos abra el navegador ya con esa URL:



Y ya podemos crear un cuaderno y trabajar con ese entorno:



localhost:8888/notebooks/Untitled.ipynb?kernel_name=pyth 120% ☆

Importar marcadores... Comenzar a usar Fir... Clases

jupyter Untitled Logout

File Edit View Insert Cell Kernel Help Trusted Python 3 (ipykernel)

Run Code

Ejemplo notebook

```
In [1]: print("¡Hola Mundo!")
```

¡Hola Mundo!

```
In [ ]:
```

Para salir pulsar **Ctrl-C** en el terminal.

9. Enlaces

- [A Guide to Python's Virtual Environments](#)
- Uso de jupyter online: <https://jupyter.org/try>
- FastAPI: <https://fastapi.tiangolo.com/>
- Google Colab: <https://colab.research.google.com/>
- Python requests. La librería para hacer peticiones http en Python: <https://j2logo.com/python/python-requests-peticiones-http/>